

DBS: Assignment 01

Date: _____

Answer#01:

a. Problems with File system Data management:

Following are few of the problems that are common.

- Data Redundancy & Inconsistency: same data stored in multiple files, which may lead to mismatches.
- Difficulty in Access: Each query needs custom programs, no general querying facility can be used.
- lack of isolation: programs are dependent on how data is physically stored.
- Integrity issues: no built-in way to enforce rules like "Age > 0".
- Security concerns: file-systems don't support fine-grained access control.
- Concurrent Access issues: multiple users updating files may cause conflicts.

b. Usage of composite primary Key:

Consider the following table: 'CourseEnrollment'

StudentID	CourseID	Grade
101	CS101	A
101	MA101	B+
102	CS 101	B

Composite Primary Key → (StudentID, CourseID)

↳ combines two or more columns to uniquely identify each row.

In the above table, neither StudentID nor CourseID alone can guarantee uniqueness, but their combination can.

Date: _____

c. Operations performed by application program when DBMS is used:

i. Data Definition Operations:

Creating tables, modifying schema and dropping schema objects. Eg: adding a new column to store students' phone#

ii. Data Manipulation Operations:

Inserting, Updating, Deleting, Retrieving. Eg: updating a student's email address or retrieving all issued books

iii. Transaction Management:

Ensuring Atomicity, Consistency, Isolation & Durability.

Eg: If a book is issued, both the 'IssuedBooks' and 'Books' tables are updated together. If one failed, both roll back.

iv. Access Control & Security:

Ensuring that only authorized users perform certain actions

Eg: Librarian can issue books, but a student cannot directly modify the database.

v. Enforcing Constraints:

DBMS checks rules automatically.

Eg: preventing negative values for 'Copies Available.'

A DBMS reduces the burden on the application program by handling low-level details like storage, indexing, and concurrency. The program only focuses on issuing high-level queries and transactions.

d. Independence in 3-schema architectures:

Logical Data Independence: changes in the conceptual schema (adding/removing attributes or tables) require rewriting application programs.

Following is the example table of 'Student':

StudentID	Name	Email
1	Mason James	mason@email.com
2	Celeste Reeves	celeste@email.com

⇒ if the conceptual schema changes (eg: email column is removed), then all external applications that display or use student emails will stop working.

e. A key is a superkey but not vice versa:

Consider the following example table: 'Employee'

EmpID	Name	Email
1	Kyle Walker	kyle@email.com
2	Dexter Morgan	dexter@email.com

⇒ superkeys: $\{EmpID\}$, $\{Email\}$, $\{EmpID, Name\}$, $\{EmpID, Email\}$, etc

⇒ candidate keys (minimal superkeys): $\{EmpID\}$, $\{Email\}$

Every key is a superkey because it uniquely identifies records, but not every superkey is a key, because a superkey may contain extra attributes. Eg: $\{EmpID, Name\}$ is a superkey but not a key (since 'EmpID' alone is sufficient).